Arche Protocol

Audit Report

contact@bitslab.xyz

https://twitter.com/movebit_

Mon Jan 27 2025

Arche Protocol Audit Report

1 Executive Summary

1.1 Project Information

Description	Arche brings you a smarter, more reliable stablecoin, powered by the Move ecosystem. MSD is a decentralized stablecoin that empowers individuals and businesses to confidently embrace a stable digital currency
Type	DeFi
Auditors	MoveBit
Timeline	Mon Dec 16 2024 - Mon Jan 27 2025
Languages	Move
Platform	Movement
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/arche-labs/smart-contracts
Commits	987b6247a068ec2d07a29dcd6c1c98a863364903 86c7ac5382b34440e0e683221ccda08708060d5e 02519f8fe537a570b84b8bf8c743678bd70a82cb

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
COL	sources/collateral.move	805003a4a48b6818958ebf7d1542fcb9afbf9644
FRA	sources/utils/fraction.move	51669e600215acc196e0d5d2a718eb2d979067ee
ROU	sources/router.move	70f25f2d57dca3166170b5c7e520b196fa092371
ADM	sources/admin/admin.move	6016bab9c37b3b9f5b2ee320972ba33901d0eba6
MSD	sources/msd.move	3d06ee4287d47a11c39341b5d8b3d353868b22a7
REG	sources/registry.move	564dbc09ff91d39e564d494b6afc049a16e4501a
CON	sources/utils/constants.move	6f06c2b4a76efc5e33eb16ff7e2c27784a074482
COM	sources/utils/common.move	9e667457452316a75c3e09bb306b2ed74e13551e
ARO	sources/admin/admin_router.move	497874e16b19e292492e2b28f6b0c8d2ddca44c8
LOA	sources/loan.move	4fa33ff36bfe88b11a5e18677e8765351c1cdb85

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	14	14	0
Informational	1	1	0
Minor	3	3	0
Medium	3	3	0
Major	6	6	0
Critical	1	1	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the **"Testing and Automated Analysis"**, **"Code Review"** and **"Formal Verification"** strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by Arche Protocol to identify any potential issues and vulnerabilities in the source code of the Arche Protocol smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 14 issues of varying severity, listed below.

ID	Title	Severity	Status
ARO-1	Interest Rate Calculation Vacuum	Minor	Fixed
ARO-2	Admin Sets Numerator Without Checking	Minor	Fixed
COM-1	The Calculation in `price_in_usd()` is Incorrect	Major	Fixed
COM-2	Use `get_price_no_older_than()` to Customize the Price Expiration Time and Retrieve the Price	Medium	Fixed
LOA-1	The User can Withdraw most of their Collateral even when they Have an Active Borrowed Position	Critical	Fixed
LOA-2	Avoiding Interest	Major	Fixed
LOA-3	Health Factor Miscalculation Due to Debt Reduction	Major	Fixed
LOA-4	Incorrect Method of Interest Calculation	Major	Fixed
LOA-5	Incorrect Implementation of	Major	Fixed

	`update_accounting_after_repay`		
LOA-6	The `liquidate` Logic Is Incorrect	Major	Fixed
LOA-7	The Check for the User's Position Health is Missing after the Liquidation	Medium	Fixed
LOA-8	Small Positions may Have Liquidation Delay Errors	Medium	Fixed
LOA-9	Missing Check for `borrow_fee > 0`	Minor	Fixed
LOA-10	Unusable `friend` Functionality	Informational	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the Arche Protocol Smart Contract :

User

- `deposit_collateral` : User deposits collateral to secure a loan.
- `withdraw_collateral` : User withdraws their collateral.
- `borrow` : User borrows `MSD` tokens.
- `deposit_collateral_and_borrow` : User deposits collateral and borrows `MSD` tokens simultaneously.
- `repay` : User repays `MSD` tokens.
- `liquidate` : User liquidates positions with low health factor.

Admin

- `add_asset_collateral_support` : Admin adds support for a new collateral asset.
- `remove_asset_collateral_support` : Admin removes support for a collateral asset.
- `activate_asset_collateral` : Admin activates a collateral asset.
- `deactivate_asset_collateral` : Admin deactivates a collateral asset.
- `update_loan_to_value` : Admin updates the loan-to-value ratio for a collateral asset.
- `asset_update_annual_percentage_yield` : Admin updates the annual percentage yield for a collateral asset.
- `asset_update_liquidator_fee` : Admin updates the liquidation discount for a collateral asset.
- `update_asset_max_borrowable_amount` : Admin updates the maximum borrowable amount for a collateral asset.
- `add_to_whitelist` : Admin adds addresses to the whitelist.
- `remove_from_whitelist` : Admin removes addresses from the whitelist.
- `update_borrow_fee` : Admin updates the borrowing fee.
- `offer_admin_privileges` : Admin offers admin privileges.

- `cancel_admin_privileges_offer` : Admin cancels an offer for admin privileges.
- `claim_admin_privileges` : Admin claims admin privileges.
- `set_treasury_address` : Admin sets the treasury address.
- `mint_interest_spread` : Admin mints interest spread.
- `update_interest_spread_rate` : Allows the admin to update the interest spread rate for loans by specifying a new numerator (`new_interest_spread_num`) and denominator (`new_interest_spread_den`). Ensures that the caller is an admin by asserting with `assert_signer_is_admin` .

4 Findings

ARO-1 Interest Rate Calculation Vacuum

Severity: Minor

Status: Fixed

Code Location:

sources/admin/admin_router.move#187;

sources/loan.move#712

Descriptions:

After the admin sets the apy parameter in the `asset_update_annual_percentage_yield` function, the time before the `accrue_interest` function is called is short, and the interest rate calculation will be too high or too low depending on the set parameters, which will cause errors in the interest rate calculation.

```
    public(friend) fun set_annual_percentage_yield(metadata: Object<Metadata>,
apy_numerator: u128, apy_denominator: u128) acquires Infos {
        let info = borrow_info_mut(metadata);
        fraction::set(&mut info.annual_percentage_yield, apy_numerator,
apy_denominator);
    }
```

Suggestion:

It is recommended to perform an accrue_interest calculation after calling apy's setup accrued_debt .

Resolution:

This issue has been fixed. The client has adopted our suggestions.

ARO-2 Admin Sets Numerator Without Checking

Severity: Minor

Status: Fixed

Code Location:

sources/admin/admin_router.move#165,187

Descriptions:

When the admin sets `numerator` and `denominator`, they do not check that the numerator must be smaller than the denominator. If the contract calculates according to the proportion and the setting is the opposite, the calculation will be wrong.

```
    public entry fun asset_update_annual_percentage_yield(
        signer_ref: &signer,
        metadata: Object<Metadata>,
        new_annual_interest_rate_numerator: u64,
        new_annual_interest_rate_denominator: u64
    ) {
        assert_signer_is_admin(signer_ref);
        let (old_annual_interest_rate_numerator, old_annual_interest_rate_denominator) =
loan::annual_percentage_yield(metadata);
        if (old_annual_interest_rate_numerator == (new_annual_interest_rate_numerator as
u128) && old_annual_interest_rate_denominator ==
(new_annual_interest_rate_denominator as u128)) {
            return
        };
        loan::set_annual_percentage_yield(metadata, (new_annual_interest_rate_numerator
as u128), (new_annual_interest_rate_denominator as u128));
        event::emit(NewAnnualPercentageYield {
            asset: metadata,
            old_annual_interest_rate_numerator: (old_annual_interest_rate_numerator as
u64),
            old_annual_interest_rate_denominator: (old_annual_interest_rate_denominator
as u64),
            new_annual_interest_rate_numerator,
            new_annual_interest_rate_denominator,
        });
    }
```

Suggestion:

It is recommended to add a check, for example:

```
new_annual_interest_rate_numerator < new_annual_interest_rate_denominator
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

COM-1 The Calculation in `price_in_usd()` is Incorrect

Severity: Major

Status: Fixed

Code Location:

sources/utils/common.move#37-41

Descriptions:

The purpose of the `price_in_usd()` function is to calculate the value of the Move asset relative to USD. The protocol first retrieves the price from Pyth and then calculates `price_in_usd = amount * 10^expo_magnitude / price_magnitude`.

```
public fun price_in_usd(_metadata: Object<Metadata>, amount: u128): u128 {
    // fetch the price of MOVE/USD
    let price =
pyth::get_price(price_identifier::from_byte_vec(constants::move_usd_price_feed_id()));

    // convert the pyth price to the number of MOVE tokens
    let price_magnitude = i64::get_magnitude_if_positive(&price::get_price(&price));
    let expo_magnitude = i64::get_magnitude_if_negative(&price::get_expo(&price));
    let move_decimals = (coin::decimals<MOVE>() as u128);

    math128::mul_div(
      amount,
      move_decimals * math128::pow(10, (expo_magnitude as u128)),
      (price_magnitude as u128)
    )
  }
```

This calculation is incorrect. The correct formula should be `price_in_usd = amount * price_magnitude / 10^expo_magnitude`.

Suggestion:

It is recommended to modify the calculation as follows:

```
math128::mul_div(
  amount,
```

```
    (price_magnitude as u128),
    move_decimals * math128::pow(10, (expo_magnitude as u128))
)
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

COM-2 Use `get_price_no_older_than()` to Customize the Price Expiration Time and Retrieve the Price

Severity: Medium

Status: Fixed

Code Location:

sources/utils/common.move#30

Descriptions:

In the `price_in_usd()` function, the protocol calls `pyth::get_price()` to retrieve the price of Move from Pyth.

```
let price =
pyth::get_price(price_identifier::from_byte_vec(constants::move_usd_price_feed_id()));
```

In the `calculate_new_health_factor()` function, this price is used to calculate the user's collateral.

```
// adjusted collateral amount
    let adjusted_collateral_amount = math128::mul_div(
        common::price_in_usd(
            constants::move_metadata(),
            new_collateral_amount
        ),
        ltv_numerator,
        ltv_denominator,
    );
```

The issue is that in the `get_price()` function, the protocol uses the default expiration check time, which may not be optimal. https://docs.pyth.network/price-feeds/best-practices

```
  public fun get_price(state: &PythState, price_info_object: &PriceInfoObject, clock:
&Clock): Price {
    get_price_no_older_than(price_info_object, clock,
```

```
state::get_stale_price_threshold_secs(state))
  }
```

This can lead to inaccurate calculations of `adjusted_collateral_amount`, causing a position that should not be liquidated to be mistakenly liquidated.

Suggestion:

It is recommended to customize the price expiration time and use `get_price_no_older_than()` to retrieve the price. https://github.com/pyth-network/pyth-crosschain/blob/e7bf47a18e2d9a9d983214342540691c1bada52e/target_chains/sui/contracts/sourc L375

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LOA-1 The User can Withdraw most of their Collateral even when they Have an Active Borrowed Position

Severity: Critical

Status: Fixed

Code Location:

sources/loan.move#280

Descriptions:

In the `calculate_new_health_factor()` function, when `is_collateral_added` is `true`, the protocol calculates `new_collateral_amount = collateral_amount() + collateral_amount`, and when `is_collateral_added` is `false`, it subtracts `collateral_amount`.

```
// total collateral amount
    let new_collateral_amount = if (is_collateral_added) {
        (collateral_amount(metadata, user_addr) as u128) + collateral_amount
    } else {
        (collateral_amount(metadata, user_addr) as u128) - collateral_amount
    };
```

This `new_collateral_amount` is then used to calculate the user's health factor.

```
    // ensure the new health factor remains healthy
    let (new_health_factor_num, new_health_factor_denom) =
calculate_new_health_factor(metadata, signer_addr, true, true, (amount as u128), 0);
    assert!((new_health_factor_num as u128) >= (new_health_factor_denom as u128),
EINSUFFICIENT_COLLATERAL);
```

When withdrawing collateral, the protocol passes `is_collateral_added = true` to `calculate_new_health_factor()`, implying that the user has added collateral. However, this causes an error in the health factor calculation, as it incorrectly adds the `collateral_amount` when the user is actually withdrawing it. This allows the user to withdraw most of their collateral without affecting their health factor.

Suggestion:

It is recommended to pass `is_collateral_added = false` to `calculate_new_health_factor()` in the `withdraw_collateral()` function.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LOA-2 Avoiding Interest

Severity: Major

Status: Fixed

Code Location:

sources/loan.move#463-494

Descriptions:

In the `remove_borrow_amount_from_position` function, the user reduces the `borrow_amount` and `debt_share_rate` in the position through a `repay` operation. However, the `old_debt_share_rate_num` and `old_debt_share_rate_denom` are directly retrieved from the position. If the user first borrows and then delays repaying, the `debt_share_rate` in the position remains as last updated during the previous borrowing action. The interest accrued between borrowing and repaying is not accounted for, meaning the user effectively avoids paying the interest for that period.

```
// calculate the new debt share rate
let (old_debt_share_rate_num, old_debt_share_rate_denom) =
fraction::fraction(&position.debt_share_rate);
let new_debt_share_rate_num = old_debt_share_rate_num - (borrow_amount as u128);
let new_debt_share_rate_denom = old_debt_share_rate_denom - (borrow_amount as
u128);
let new_debt_share_rate = fraction::init(new_debt_share_rate_num,
new_debt_share_rate_denom);

// update the position
let new_position = Position {
    collateral_amount: position.collateral_amount,
    borrow_amount: new_borrow_amount,
    debt_share_rate: new_debt_share_rate,
    last_debt_share_updated: timestamp::now_seconds()
};
let positions = borrow_global_mut<Positions>(signer_addr);
table::upsert(
    &mut positions.table,
    object::object_address(&metadata),
```

```
    new_position
);
```

Suggestion:

It is recommended to retrieve the debt share rate through the `debt_share_rate` function instead of directly fetching it from the position.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LOA-3 Health Factor Miscalculation Due to Debt Reduction

Severity: Major

Status: Fixed

Code Location:

sources/loan.move#403-404

Descriptions:

In the liquidate() function, the code calls calculate_new_health_factor() to compute new_health_factor_num and new_health_factor_denom. The parameters passed to the calculate_new_health_factor() function include unlocked_collateral * ltv_numerator and amount * ltv_denominator. In the test file, the configuration sets ltv_numerator to 1 and ltv_denominator to 2.

Within the calculate_new_health_factor() function, the protocol uses debt_share_value - added_borrow_amount, which results in a reduction of debt. This causes the computed new_health_factor to be inaccurate.

Suggestion:

It is recommended to avoid multiplying ltv_numerator and ltv_denominator.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LOA-4 Incorrect Method of Interest Calculation

Severity: Major

Status: Fixed

Code Location:

sources/loan.move#572-623

Descriptions:

In the `accrue_interest` function, interest is calculated and added to `total_debt`. However, this method accumulates interest on the updated `total_debt`, resulting in compounding interest during subsequent calculations.

For example, assume the initial `debt` is 100 with a daily interest rate of 10%. Over two days, the correct total debt should be 120 (100 + 10% * 100 + 10% * 100). In this protocol, however, if `accrue_interest` is called at the end of the first day, the `total_debt` becomes 110. On the second day, the `total_debt` is calculated as `110 + 110 * 10%`, resulting in 121, which is higher than the correct value. The more frequently `accrue_interest` is called, the higher the accumulated interest, which also causes a mismatch with the interest users owe on their positions. The user's borrowing interest is only updated during the `borrow` operation, which causes the user's interest to be updated less frequently than the protocol's total interest. As a result, the total debt recorded by the protocol exceeds the aggregate debt of all users.

```
// calculate the debt interest
let (apy_numerator, apy_denominator) = annual_percentage_yield(metadata);
let accrued_debt = math128::mul_div(
    total_debt,
    (apy_numerator as u128) * (elapsed_time as u128),
    (apy_denominator as u128) * (constants::seconds_per_year() as u128)
);

// calculate the interest spread (protocol revenue portion)
let (spread_numerator, spread_denominator) = interest_spread_rate(metadata);
let interest_spread = math128::mul_div(accrued_debt, spread_numerator,
spread_denominator);
```

```
let info = borrow_info_mut(metadata);

// update the accrued interest in total debt
info.total_debt = total_debt + accrued_debt;

// add the interest spread to the protocol's revenue
info.total_interest_spread = info.total_interest_spread + interest_spread;
```

Suggestion:

It is recommended to adjust the interest logic of the protocol to ensure that the total interest of the protocol aligns with the interest accrued by the users.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LOA-5 Incorrect Implementation of `update_accounting_after_repay`

Severity: Major

Status: Fixed

Code Location:

sources/loan.move#540-550

Descriptions:

In the `update_accounting_after_repay` function, the following logic is used:

```
let user_accrued_interest = user_accrued_interest(metadata, signer_addr);
let borrow_amount_to_remove = if ((amount as u128) > user_accrued_interest) {
    (amount as u128) - user_accrued_interest
} else {
    (amount as u128)
};
```

If `amount > user_accrued_interest`, the value passed to the `remove_borrow_amount_from_position` function is `amount - user_accrued_interest`. This decreases the `borrow_amount` of the position and updates the new debt share rate.

However, the calculation of the new debt share rate should be based on the full `amount`, not `amount - user_accrued_interest`.

This discrepancy causes the user's debt reduction to be less than expected.

Suggestion:

Fix the issue by ensuring that the new debt share rate is calculated using the full `amount` rather than `amount - user_accrued_interest`.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LOA-6 The `liquidate` Logic Is Incorrect

Severity: Major

Status: Fixed

Code Location:

sources/loan.move#385

Descriptions:

In the `liquidate` function, the `new_debt` must be greater than or equal to `max_borrowable_amount`. However, the calculation of `new_debt` subtracts the repayment `amount`, which implies that the debt after liquidation must still exceed `max_borrowable_amount`. This logic is evidently flawed.

The correct logic should ensure that the debt after liquidation is less than `max_borrowable_amount`, placing the user's position in a healthy state.

```
let new_debt = debt_share_value(metadata, at_risk_address) - (amount as u128);
let max_borrowable_amount = max_borrowable_amount_from_collateral(
    metadata,
    common::price_in_usd(
        constants::move_metadata(),
        (collateral_amount(constants::move_metadata(), at_risk_address) as u128)
    )
);
assert!(new_debt >= max_borrowable_amount,
ECANNOT_LIQUDATE_MORE_THAN_SURPLUS);
```

Suggestion:

Change the `>=` comparison to `<` to ensure that the debt after liquidation is less than `max_borrowable_amount`.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LOA-7 The Check for the User's Position Health is Missing after the Liquidation

Severity: Medium

Status: Fixed

Code Location:

sources/loan.move#395-400

Descriptions:

The purpose of the `liquidate()` function is to liquidate the excess debt of the `at_risk_address`. However, after the liquidation, the protocol does not check the health status of the position to ensure that the position is healthy after the liquidation.

```
// accounting: remove the collateral from the risk address position
    let liquidated_amount = amount + liquidator_fee + protocol_liquidation_fee;
    let unlocked_collateral = (common::price_in_move(metadata, (liquidated_amount as u128)) as u64);
    update_accounting_after_collateral_withdraw(metadata, at_risk_address, unlocked_collateral);

    // burn the repaid amount
    msd::burn(&object_signer(), liquidator_addr, amount);
```

Suggestion:

It is recommended to check the health status of the user's position.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LOA-8 Small Positions may Have Liquidation Delay Errors

Severity: Medium

Status: Fixed

Code Location:

sources/loan.move#371 505

Descriptions:

The contract only checks the health of individual positions and calculates the total debt. When prices fall, they are greatly affected. The contract liquidator will prioritize liquidating high-yield positions. If the yield of an individual position is extremely small, causing the position to completely fall below the price, the position owner will be in a liquidated penetration state and will not execute "repay", and the liquidator will not execute the negative-yield "liquidate", but the bad debt position will continue to affect the entire system and accrue interest.

```
    assert!(!health_factor_is_at_or_above_minimum(metadata, at_risk_address),
ENOT_UNDERCOLLATERALIZED);
```

Suggestion:

It is recommended to set up a `liquidate_bad_debt` function to handle possible bad debts to avoid system damage.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LOA-9 Missing Check for `borrow_fee > 0`

Severity: Minor

Status: Fixed

Code Location:

sources/loan.move#307

Descriptions:

In the `calculate_borrow_fee()` function, the protocol calculates the `borrow_fee_to_pay` as `amount * borrow_fee_num / borrow_fee_denom`.

```
fun calculate_borrow_fee(
        signer_ref: &signer,
        metadata: Object<Metadata>,
        amount: u64
    ): u64 {
        let (borrow_fee_num, borrow_fee_denom) = admin::borrow_fee();

        // return early if the borrow fee is 0
        if (borrow_fee_num == 0) { return 0 };

        // calculate the borrow fee to pay
        let borrow_fee_to_pay = (math128::mul_div((amount as u128), borrow_fee_num,
borrow_fee_denom) as u64);

        // emit event
        event::emit(BorrowFeeCharged {
            metadata: object::object_address(&metadata),
            borrower: signer::address_of(signer_ref),
            amount: borrow_fee_to_pay
        });

        borrow_fee_to_pay
    }
```

However, the protocol does not check if the `borrow_fee > 0`. Malicious users could initiate multiple borrow transactions, each borrowing a small amount of assets, thereby evading the fee.

```
 // if signer is not whitelisted, ensure the new balance is below the maximum borrowable amount
        assert!(!exceeds_max_borrowable_amount(metadata, signer_addr, amount), EMAX_BORROWABLE_AMOUNT_EXCEEDED);

        // calculate the borrow fee amount
        let borrow_fee = calculate_borrow_fee(signer_ref, metadata, amount);

        // calculate the amount to borrow after fee
        let amount_with_fee = amount + borrow_fee;
```

Suggestion:

It is recommended to add a check to ensure that `borrow_fee > 0`.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LOA-10 Unusable `friend` Functionality

Severity: Informational

Status: Fixed

Code Location:

sources/loan.move#717-721;

sources/admin/admin.move#100-103

Descriptions:

The contract contains two functions with `public(friend)` visibility, but they are not invoked in other modules, rendering them unusable. For example, the `set_interest_spread` and `update_liquidation_fee` functions:

```
/// Update the protocol liquidation fee
public(friend) fun update_liquidation_fee(numerator: u128, denominator: u128) acquires
LiquidationFee {
    let liquidation_fee = borrow_global_mut<LiquidationFee>(@arche);
    fraction::set(&mut liquidation_fee.amount, numerator, denominator);
}

/// Helper function to set new interest spread
public(friend) fun set_interest_spread(metadata: Object<Metadata>,
interest_spread_numerator: u128, interest_spread_denominator: u128) acquires Infos {
    let info = borrow_info_mut(metadata);
    fraction::set(&mut info.interest_spread, interest_spread_numerator,
interest_spread_denominator);
}
```

Suggestion:

Add invocations for these functions in the `router` module to enable their usage.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.